SDG BLOCKCHAIN ACCELERATOR

Technical Architecture Document

1. Project Information

- **Project Name:** Digital Infrastructure for Green Energy Exports / Fuel Switch x UNDP BiH Pilot - Cohort 2
- **Challenge & UNDP Office:** Bosnia & Herzegovina Country Office

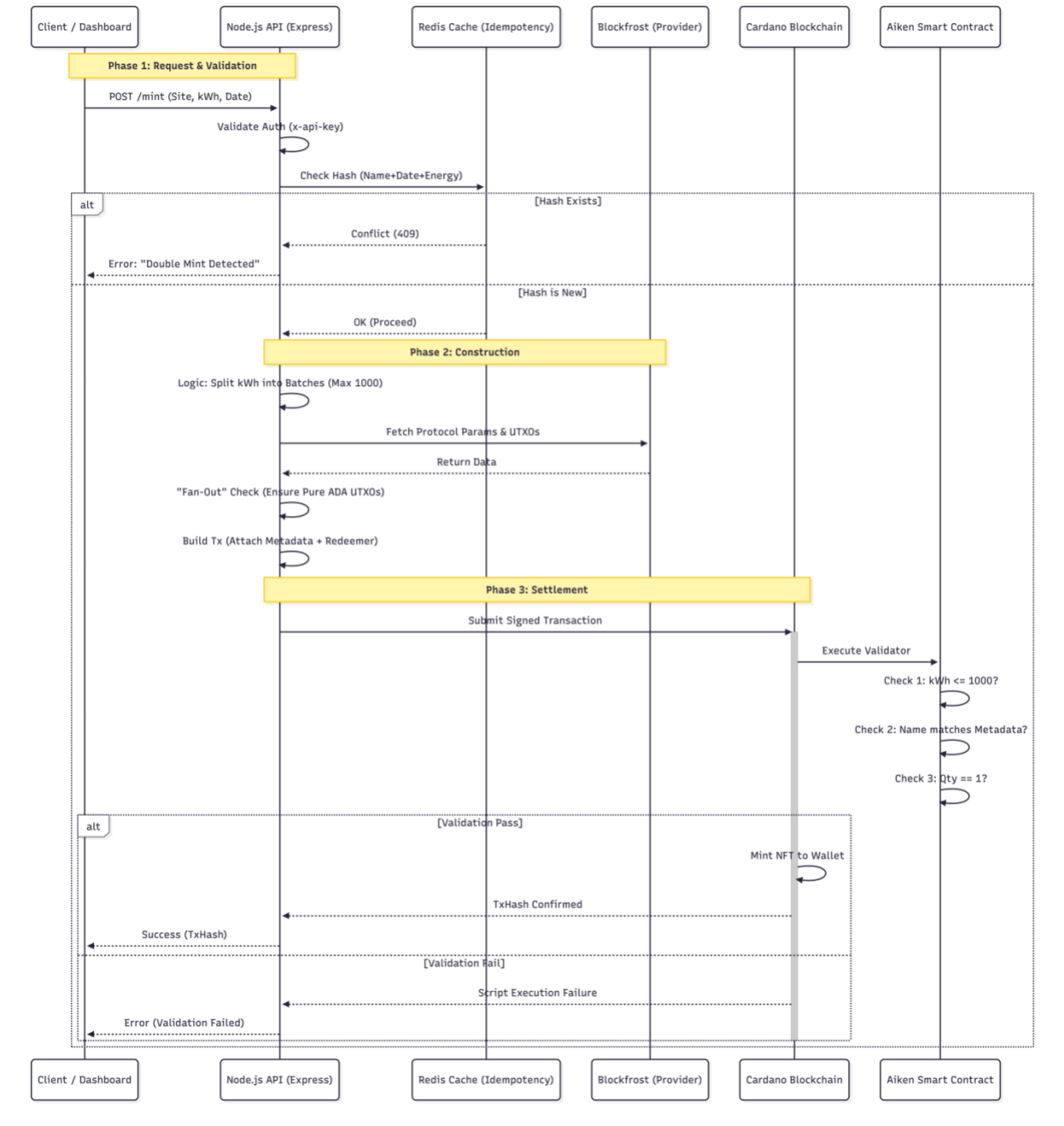
- **Document Version: 1.0.0**

2. Overview

- The FuelSwitch REC Manager is a "Plutus V3" enabled Node.js service that acts as a bridge between solar data aggregators and the Cardano blockchain. It solves the problem of "greenwashing" and double-counting in renewable energy markets by tokenizing verified generation data into immutable Renewable Energy Certificates (RECs).

- The system enforces a strict 1-to-1 correlation between physical energy (kWh) and digital assets (NFTs) using on-chain validation. It features a high-availability architecture capable of handling concurrent minting operations via intelligent UTXO management.

3. System Architecture Diagram



4. Blockchain Design

- **Smart Contracts:**
    - Validator script(s) with purpose :
        - rec_nft_mint_policy (Aiken / Plutus V3): Acts as the immutable gatekeeper for the ecosystem. Its primary purpose is to enforce data integrity, ensuring that the Asset Name on-chain matches the metadata provided (preventing "spoofing") and strictly limiting energy batches to a maximum of 1000 kWh per token.
    - Datum structure :
        - None. As this is a Minting Policy (not a Spending Validator), it does not utilize a Datum. All validation logic relies on the Transaction Context and the Redeemer.
    - Redeemer structure :
        - Custom Data Type: Action with two modes:
            - MintBatch: Contains the RecDetails struct (Site Name, Date, kWh, Batch ID) used to validate the asset generation.
            - BurnRec: A simple signal to allow asset destruction (retirement), with logic to ensure no positive values are minted during a burn.
- **UTxO Model Usage:**
    - How inputs and outputs are handled:
        - Inputs: The system utilizes a "Fan-Out" strategy to manage concurrency. It explicitly filters for "Pure ADA" UTXOs (containing only Lovelace) to use as collateral and inputs, preventing "Collateral Contains Non-ADA" errors.
        - Outputs: Transactions are chunked to contain a maximum of 15 assets per transaction to adhere to protocol size limits.
    - Examples of typical transaction flows:
        - Minting Flow: The backend splits a large energy total (e.g 3450 kWh) into smaller chunks , Selects available Pure ADA inputs , Constructs a transaction minting 4 distinct NFTs (3x 1000 kWh, 1x 450 kWh) , Submits to ledger.
- **Token Management:**
    - On-chain asset minting/burning rules:
        - Minting:

            - Cap Enforcement: kw must be <= 1000. If kw > 1000, the script fails.
            - Naming Convention: Asset Name is deterministically generated: REC-[Site]-[Date]-[kWh]-[BatchID] (Hex Encoded). If the metadata in the redeemer doesn't match this pattern, minting is rejected.
            - NFT Standard: Quantity must strictly equal 1.
        - Burning: The validator allows burning but checks that the minted value is negative (or zero) to prevent exploiting the burn redeemer to mint new tokens.
    - Tracking ownership via datum:
        - Ownership is native to the Cardano eUTXO model. The specific UTXO holding the REC NFT represents current ownership. The batch_id inside the Asset Name ensures every token is unique, allowing for granular tracking of specific energy batches.

- **Security Considerations:**
    - Signature checks (authorized wallets):
        - Access is primarily controlled off-chain via the API Key (x-api-key) which gates access to the operational hot wallet.
    - Datum/redeemer validation:
        - The smart contract performs deep validation of the Redeemer fields. It mathematically verifies that the Asset Name == Hash(Site + Date + kWh + BatchID). This makes it impossible to mint a token named "Solar" while providing "Coal" data in the redeemer.
    - Replay protection / double-spend prevention:
        - Off-Chain: Redis Idempotency checks a unique hash (Name + Date + Energy) before building the transaction to prevent double-minting due to network lag.
        - On-Chain: The standard UTXO consumption model prevents replay attacks. Additionally, the unique Batch ID ensures that once a specific batch is minted, a duplicate cannot be created with the same ID.

Example:

- Validator Enforcement: The validator strictly enforces that the Asset Name is cryptographically derived from the input metadata (preventing spoofing) and that the energy value does not exceed 1000 kWh per token.
- Datum/Redeemer: No Datum is utilized; instead, the Redeemer contains the full RecDetails payload (Site Name, Vintage Date, kWh, Batch ID) for validation.
- Execution Check: The script validates the Action type (MintBatch or BurnRec) ensures that "Burn" actions cannot be exploited to mint positive token amounts.

5. Data Flow & Transaction Lifecycle

1. User Request:
    - The client/dashboard submits a JSON payload to the API (POST /mint) containing the Site Name, Energy Amount (kWh), and Vintage Date .
    - Off-Chain Validation (Idempotency): The Node.js server immediately calculates a unique hash of the request and checks Redis. If the hash exists, the request is rejected (409 Conflict) to prevent double-minting; otherwise, it proceeds .
2. Batching & Construction:
    - The server automatically splits the total energy (e.g., 3450 kWh) into standardized 1000 kWh batches.
    - It selects "Pure ADA" UTXOs (via "Fan-Out" logic) to prevent collateral errors and constructs the transaction using @lucid-evolution/lucid.
3. On-Chain Verification: The transaction reaches the Aiken validator script (rec_nft_mint_policy), which performs three critical checks:
    - Anti-Spoofing: Verifies that the Asset Name matches the hashed metadata in the Redeemer.
    - Cap Enforcement: strict check that kWh <= 1000.
    - NFT Standard: strict check that Quantity = 1.

4. Settlement & Persistence: If valid, the REC NFTs are minted to the operational wallet. The unique Transaction Hash is returned to the user, and the system updates Redis to mark the batch as complete.

6. Off-chain Components

- Backend services (if any):
    - Node.js API (Express): The core service that acts as the "bridge" between solar data and the blockchain. It handles critical business logic including Batch Splitting (breaking large energy amounts into 1000 kWh chunks) and UTXO Management (using "Fan-Out" logic to split large UTXOs into smaller "Pure ADA" inputs to support concurrency).
    - Redis Database: A persistence layer used for state management. It stores unique hashes of every request to provide Idempotency, ensuring that network lag does not result in double-minting.
- Integration with Cardano node / testnet:
    - Library: The system uses @lucid-evolution/lucid to construct transactions and interact with the ledger.
    - Network: Configured for the Cardano Preprod Network for User Acceptance Testing (UAT), with environment variables set to switch to Mainnet for production.
- Oracles / data providers:
    - Blockfrost API: The primary provider used to query the blockchain (fetching protocol parameters and UTXOs) and submit signed transactions.
    - Provider Agnosticism: The architecture is designed to be provider-agnostic; in a disaster scenario, Blockfrost can be swapped for Kupmios or a local Cardano Node with minimal code changes.
- Dashboard or reporting tools:
    - Management UI: A frontend interface connected to the backend endpoints (/mint, /burn). It provides real-time feedback to the operator, displaying "Processing" states (approx. 30s) and returning success messages with Transaction Hashes and links to Cardanoscan for verification .

7. Sandbox/Testnet Results

- Environment: Cardano Preprod Network (Testnet Magic) Compiler: Aiken v1.1.19 (Plutus V3):
- A. Transaction Performance Matrix
- The following summarizes the on-chain execution metrics for the core smart contract functions. Note the efficiency of Plutus V3 usage in the "Burn" and "Mint" validators.

Example:

Mint:

```
Server running on port 3000
Initializing Lucid...
Performing UTXO Health Check...
Found 20 Pure ADA UTXOs.
Wallet health sufficient. Proceeding.
Processing Chunk 1/1...
Chunk 1 submitted: 9f1fa7585242ac040c2a03f7e14a65241797930497aa4c9fc316066472137df3
```

```
macbook@Macbooks-MacBook-Pro webapp_blockchain % curl -X POST http://localhost:3000/mint \
  -H "Content-Type: application/json" \
  -H "x-api-key: FSDCYKJHGYU78SDFGHJK23456789TYUI" \
  -d '{
    "metadata": {
      "name": "Solektro",
      "energy_kwh": 2200,
      "vintage_start": 1763712343,
      "vintage_end": 1763712350
    }
  }'
{"status":"success","txHashes":["9f1fa7585242ac040c2a03f7e14a65241797930497aa4c9fc316066472137df3
"],"policyId":"1eb188b6fa918272aa51b85364345b2c0f3083788c738c4f4a746792"}%
```

Burn part of a batch:

```
Attempting to Burn: 5245432d536f6c656b74726f2d69201d572d000003e82d00000002
```

```
  -H "Content-Type: application/json" \
  -H "x-api-key: FSDCYKJHGYU78SDFGHJK23456789TYUI" \
  -d '{
  "metadata": {
    "name": "Solektro",
    "energy_kwh": "1000",
    "vintage_start": 1763712343,
    "batch_id": "2"
    }
  }'
{"status":"burned","txHash":"9b4ea4a01220d16cfc1d4aed27c768aaacbd0ed680c3d499c108e6ef973fba3e","a
sset":"1eb188b6fa918272aa51b85364345b2c0f3083788c738c4f4a7467925245432d536f6c656b74726f2d69201d57
2d000003e82d00000002"}%
```

Burn Batch:
(Note the skipped transaction is due to the one we have burned above already)

```
Server running on port 3000
Initializing Lucid...
Performing UTXO Health Check...
Found 20 Pure ADA UTXOs.
Wallet health sufficient. Proceeding.
Processing Chunk 1/1...
Chunk 1 submitted: 9f1fa7585242ac040c2a03f7e14a65241797930497aa4c9fc316066472137df3
Attempting to Burn: 5245432d536f6c656b74726f2d69201d572d000003e82d00000002
Fetching history for Tx: 9f1fa7585242ac040c2a03f7e14a65241797930497aa4c9fc316066472137df3...
Burning 2 assets. (Skipped 1 unavailable)
```

```
macbook@Macbooks-MacBook-Pro webapp_blockchain % curl -X POST http://localhost:3000/burn-batch \
  -H "Content-Type: application/json" \
  -H "x-api-key: FSDCYKJHGYU78SDFGHJK23456789TYUI" \
  -d '{
    "txHash": "9f1fa7585242ac040c2a03f7e14a65241797930497aa4c9fc316066472137df3"
  }'
{"status":"success","message":"Burned 2 assets","txHash":"237f1e87aa1a52b68608245829507e4d7b689be
fab37e81b78b511cc0190c2c8"}%
macbook@Macbooks-MacBook-Pro webapp_blockchain %
```

8. Tools and Environments Used

1. Smart Contract & Compiler

- Language & Compiler: Aiken v1.1.19
- CLI Tools Used:
  - aiken build: Used to compile the contract and generate the plutus.json blueprint.
  - aiken check: Used to run the native unit test suite (e.g., mint_01_batch_success, mint_02_fail_kw_too_high).
  - aiken blueprint policy: Used to derive the Policy ID from the compiled script.
- Plutus Version: Plutus V3

2. Blockchain Provider & Node Access

- Primary Provider: Blockfrost API
    - Instead of running a local cardano-node instance on the backend server, the architecture relies on Blockfrost for chain query and transaction submission.
- Fallback/Alternative: Kupmios (Kupo + Ogmios) or Local Cardano Node 5. The Security and Threat Mitigation Report notes that the system is architected to be provider-agnostic. In a disaster scenario (e.g Blockfrost downtime), the backend can be reconfigured to use a local Node or Kupmios setup.

3. Off-Chain Integration & Backend

- Blockchain Library: @lucid-evolution/lucid
    - Used for transaction construction, UTXO management (Fan-Out strategy), and signing. It handles the "Big Endian Hex" conversions required to match Aiken's data structures.
- Runtime Environment: Node.js (with Express framework).
- Database/Cache: Redis (via ioredis)
- Used for Idempotency (preventing double-mints) and managing the "Fan-Out" state.

4. Deployment Environment (Testnet)

- Network: Cardano Preprod
    - All User Acceptance Testing (UAT) and final sign-offs are conducted on the Preprod network to simulate mainnet conditions without risking real assets.
- Hosting Infrastructure: Render
    - The Node.js service is deployed to a cloud provider capable of sustaining persistent connections to Redis and the Blockfrost API.

9. Remaining Considerations / Next Steps

1. Pending Architecture Changes

    - Validator Logic Update (Aiken):
        - Objective: Modify rec_nft_mint.ak to support fungible minting.
        - Change: Remove the quantity == 1 enforcement. Update the logic to allow quantity > 1 (representing total kWh or total RECs).
        - Naming Convention: Simplify the Asset Name. Instead of REC-[Site]-[Date]-[kWh]-[BatchID], the new standard should likely be REC-[Site]-[Date].
        - Result: All energy generated by Site A on Date B becomes fungible. If they generate 5,000 kWh, you mint 5,000 units of the same asset.
    - Backend Loop Removal:
        - Objective: Deprecate the "Transaction Chunking" logic.

        - Change: Remove the code that splits energy into 1,000 kWh chunks and loops through arrays of 15.
        - New Flow: Receive 3450 kWh, Build 1 Transaction, Mint 3,450 tokens of REC-[Site]-[Date].

2. Performance Improvements

    - Eliminate "Fan-Out" Dependency:
        - Since we no longer need to submit 10+ transactions in parallel for a single day's generation, the complex "Fan-Out" (UTXO splitting) strategy is less critical. You can reduce the "pure ADA" UTXO buffer significantly (e.g from 30 UTXOs to 5), freeing up locked ADA.

3. Security Audit Steps (Updated for FTs)

    - Inflation Vulnerability Check:
        - Risk: With NFTs, minting 1 million tokens meant 1 million transactions (impossible/expensive). With FTs, minting 1 million tokens is one cheap transaction.
        - Audit: Verify the Smart Contract strictly validates quantity == redeemer.total_kwh. Ensure the redeemer data (signed by the oracle/backend) matches the mint quantity exactly.
    - Burn Logic Verification:
        - Test: Ensure the "Burn" validator allows burning partial amounts (e.g., retiring 500 kWh out of a 5,000 kWh balance) rather than requiring the whole asset to be destroyed.